

E37 Order API Reference

Everything we currently know about pulling order data out of the E37 webshop platform: the new order flow report, the existing Triton Admin REST API, and the status webhook. Compiled from E37's own documentation and correspondence so the integration team can start designing before the API key is in hand.

Compiled **3 September 2026** Platform **E37 Triton Admin 1.0** Vendor contact **support@e37.se**

Where we stand

REPORT ENDPOINT

LIVE Delivered by E37 on 27 Aug 2026. URL shape and JSON fields confirmed from screenshots.

REST API

DOCUMENTED Public OpenAPI spec at admin3.e37.se/docs. Four endpoints, one webhook payload.

API KEY

BLOCKED Creating a key in E37 Admin currently fails with an error. Tommy is in contact with E37.

HISTORICAL BACKFILL

PENDING E37 offered to discuss one-off exports of older order data as a next step.

Nothing in this document has been exercised against a live endpoint yet. Items marked **UNVERIFIED** are inferred and need a real call or a confirmation from E37.

Two APIs, one key

E37 exposes order data through two separate mechanisms that share the same API key but authenticate differently.

MECHANISM	PURPOSE	AUTH	FORMAT
Order flow report <code>/api/reports/orderflow</code>	Lightweight list of orders completed in a time window. Built for polling, monitoring and dashboards.	API key as <code>key</code> query parameter	JSON (also table and file downloads from the UI)
Triton Admin REST API <code>/api/orders/{id}</code> and friends	Full detail for one order: customer, addresses, payment, delivery, rows. Also marketplace order creation and gift cards.	HTTP Basic: webshop ID as username, API key as password	JSON
Order status webhook	Push notification when an order's status changes, with tracking data.	Registered in E37 Admin (outbound from E37)	JSON POST to our URL

Fredrik Karlsson at E37 explicitly recommends combining the first two: poll the report for new order numbers, then call the REST API per order for the details the report does not carry.

Authentication

Creating the key

The OpenAPI spec says only that a key is created "in your webshop's back-end", meaning E37 Admin at admin3.e37.se. The exact menu location is not documented in E37's public help centre. Christian Ekman at WeSports created a key for the Qlik feeder in December 2025, so the procedure has worked before. Key creation currently returns an error; E37 has been asked.

Treat the key as a password. Store it in a secret store or environment variable, never in a script, ticket or chat.

Report endpoint: query parameter

```
GET https://admin3.e37.se/api/reports/orderflow?account=cykloteket&key=<API KEY>&...
```

The key travels in the URL. Over HTTPS it is encrypted in transit, but it will land in E37's web server logs, Cloudflare logs, browser history and anywhere the URL is pasted. Keep the key out of shared documents and mask it in our own logs.

REST API: HTTP Basic

Username is the webshop ID, password is the API key, base64-encoded together.

```
Authorization: Basic base64(<WEBSHOP ID>:<API KEY>)
```

Recommendation to raise with E37. Since the REST API already accepts the key in an Authorization header, ask E37 to let the report endpoint accept the same Basic auth, keeping the query parameter only for link-based tools such as Excel or Power BI. This is a small change on their side and removes the key from URL logs.

Order flow report

Announced by E37 on 27 August 2026 as the "orderflödesrapport". It is a report in E37 Admin that also has a URL for automated calls. In the UI it lives under **E-handel** → **Rapporter**, report type **Orderrapport – orderflödesrapport**. The button **Skapa länk till rapport** shows the URL for the current settings.

Request

Example URL exactly as E37's screenshot shows it, with the key placeholder:

```
https://admin3.e37.se/api/reports/orderflow
?account=cykloteket
&key=<INSERT API KEY>
&dateInterval=2026-08-26+18%3a30%2c2026-08-26+19%3a30
&orderTimestampMode=completed
```

PARAMETER	VALUE	NOTES
account	Shop account slug, e.g. cykloteket	One E37 account can host several sites (shop-in-shop). Whether one key covers all accounts is UNVERIFIED .
key	API key	Created in E37 Admin.
dateInterval	YYYY-MM-DD HH:MM, YYYY-MM-DD HH:MM	Start and end separated by a comma. URL-encoded, space becomes + , colon %3a , comma %2c . Inclusive/exclusive boundaries are UNVERIFIED .
orderTimestampMode	completed	Filter on the time the payment provider confirmed the order (Klarna, Svea callback etc.), not the creation time. This is what makes gap-free polling possible. Other values presumably exist for creation time; UNVERIFIED .

The UI also offers filters for order status, excluded order status and site, plus a file type selector (Json shown). Whether these appear as extra query parameters

when included in the generated link is **UNVERIFIED** . Generate a link with them set and compare.

Response, minimal column set

A JSON object with a `rows` array. Each row:

FIELD	EXAMPLE	MEANING
<code>order_id</code>	1189437	E37 order number. Use it for <code>GET /orders/{id}</code> .
<code>order_timestamp</code>	2026-08-26T18:30:13	Order time. No timezone offset in the string; almost certainly Europe/Stockholm local time. UNVERIFIED Fractional seconds appear on some rows (<code>18:35:00.75</code>).
<code>total_sum_excl_vat</code>	862.4000	Order total excluding VAT, four decimals.
<code>total_sum_incl_vat</code>	1078.0000	Order total including VAT.
<code>currency</code>	SEK	Also seen: NOK, DKK.
<code>country</code>	SE	ISO country code.
<code>country_name</code>	Sverige	Localised country name.
<code>language</code>	SV	Also seen: NO, DA.
<code>person_type</code>	Privatperson	Customer type as text. Company orders presumably differ; UNVERIFIED .
<code>site_id</code>	4	Site (shop-in-shop) numeric ID. See Known shops and sites .
<code>site_name</code>	Bikester SE	Site display name.

Optional extra columns

Tick these in the report UI before generating the link. Exact JSON field names are **UNVERIFIED** until we see a response.

- Customer number and basic customer details
- Amount paid with gift card and remaining amount to pay
- Shipping cost
- Order status
- Other status: Garp (Spobik) ERP sync status and any order error code
- Payment method
- External marketplace

Polling recipe

E37's recommended cadence: fetch every 30 minutes, two minutes after the half hour, always asking for the previous closed half-hour window. Because the filter is on completion time, orders that were created earlier but paid late fall into the window in which they were actually completed, so nothing is missed and nothing is fetched twice.



Practical additions on our side:

- Persist the last successfully fetched window end. On failure, retry the same window rather than skipping ahead.
- Upsert on `order_id` so a re-run of a window is harmless.
- Run the schedule in Europe/Stockholm time until E37 confirms the timestamp timezone.
- Alert when a shop returns zero rows during trading hours for longer than expected. This is the monitoring use case E37 had in mind.

First manual call

PowerShell, with the key in an environment variable:

```
$env:E37_KEY = "paste-key-here"
$from = "2026-09-02+08%3a00"
$to = "2026-09-02+08%3a30"
curl.exe "https://admin3.e37.se/api/reports/orderflow?account=cykloteket&key=$env:E37_KEY&dateInterval=$from%2c$to&orderTimestampMode=completed"
```

Bash:

```
export E37_KEY="paste-key-here"
curl -s "https://admin3.e37.se/api/reports/orderflow?account=cykloteket&key=${E37_KEY}&dateInterval=2026-09-02+08%3a00%2c2026-09-02+08%3a30&orderTimestampMode=completed" | jq '.rows | length'
```

Triton Admin REST API

Source: OpenAPI 3 spec "Triton Admin 1.0", published at admin3.e37.se/docs.

Rendered with Stoplight.

Base URLs

SERVER	BASE URL
web02	<code>https://admin2.e37.se/api</code>
web03	<code>https://admin3.e37.se/api</code>

Each webshop lives on one of these servers. Fredrik's links and the report screenshots all point at web03, so WeSports shops are presumably on admin3. Confirm per shop.

GET `/orders/{id}`

Returns full information about one order. Path parameter `id` is the E37 order number (integer). Responses: 200 with an `Order` object, 404 if not found.

```
curl -u "<WEBSHOP ID>:<API KEY>" https://admin3.e37.se/api/orders/1189437
```

Order object

FIELD	TYPE	DESCRIPTION
<code>id</code>	integer	E37 order number
<code>timestamp</code>	date-time	When the order was created (e.g. <code>2022-01-14T12:07:39.827</code> , no offset)
<code>synced</code>	boolean	Whether the order has been synced to the ERP
<code>erpOrderNumber</code>	string	Order number in the ERP, null until synced
<code>languageCode</code>	string	e.g. <code>sv</code>
<code>currencyCode</code>	string	e.g. <code>sek</code>
<code>externalOrder</code>	object	For marketplace orders: <code>source.id</code> , <code>source.name</code> , <code>externalOrderNumber</code> , <code>timestamp</code>
<code>site</code>	object	<code>id</code> (string) and <code>name</code> of the site the order was placed on
<code>orderStatus</code>	object	<code>id</code> (e.g. <code>NEWORDER</code>) and <code>name</code>
<code>customer</code>	object	<code>customerNumber</code> , <code>type</code> (1 individual, 2 company), <code>newCustomer</code> , <code>acceptsMarketing</code> , <code>email</code> , <code>phone</code> , <code>mobilePhone</code> , <code>invoiceAddress</code> , <code>deliveryAddress</code> . Addresses have <code>name</code> , <code>address1</code> , <code>address2</code> , <code>zipCode</code> , <code>city</code> , <code>countryCode</code> .
<code>payment</code>	object	<code>paymentMethod</code> (e.g. "Klarna Checkout V3"), <code>reference</code> , <code>status</code> (ACTIVATED, CANCELLED, NOT_ACTIVATED), <code>feeExclVat</code> , <code>feeInclVat</code> , <code>feeVatAmount</code>
<code>delivery</code>	object	<code>deliveryMethod</code> , fee fields, <code>deliveryDate</code> , <code>pickupDate</code> , <code>tms</code> (<code>sent</code> , <code>sentTimestamp</code> , <code>serviceID</code>)
<code>pickupLocation</code>	object	<code>id</code> , <code>type</code> (0 none, 1 transport company, 2 matrix, 3 location, 4 reseller, 5 time slot), <code>name</code> , <code>address</code> , <code>zipCode</code> , <code>city</code>
<code>ipAddress</code>	string	Customer IP
<code>message</code>	string	Message from customer
<code>totalSumInclVat</code> , <code>totalSumExclVat</code> , <code>vatSum</code>	number	Order totals
<code>totalWeight</code>	number	Total weight (grams in the example)
<code>rows</code>	array	Order rows, see below

Order row

FIELD	DESCRIPTION
<code>id</code>	Order row ID
<code>sku</code> , <code>supplierSKU</code> , <code>manufacturerSKU</code>	Product identifiers
<code>name</code> / <code>title</code>	Product name. The schema says <code>name</code> , the example says <code>title</code> . Expect either; UNVERIFIED .
<code>variant</code> / <code>matrices</code>	Variant text, e.g. "Grön, Medium". Same schema-versus-example mismatch as above.
<code>brand</code> / <code>trademark</code>	Brand, e.g. "Scott". Same mismatch.
<code>quantity</code> , <code>unit</code> , <code>weight</code>	Quantity, unit ("st"), weight per unit
<code>vatPercentage</code>	VAT rate, e.g. 25
<code>priceInclVat</code> , <code>priceExclVat</code>	Unit price actually paid
<code>sumInclVat</code> , <code>sumExclVat</code>	Row totals actually paid
<code>regularPriceInclVat</code> , <code>regularPriceExclVat</code> , <code>regularSumInclVat</code> , <code>regularSumExclVat</code>	Undiscounted price and sums. Discount per row is the difference.
<code>inStock</code> , <code>stockInfo</code>	Stock state at order time
<code>returnQuantityAccepted</code> , <code>returnQuantityPending</code>	Returns
<code>isPackageRoot</code> , <code>subRows</code>	Bundle structure
<code>additionalInfo</code> , <code>attachment</code>	Free text and file attachment

Note what is **not** in the Order object: discount or campaign rows as separate lines, status history, and delivery promise. Those were on Niklas's wish list for the one-off export and need to come from E37 directly or from the status webhook over time.

GET `/orders/{id}/status`

Returns only the current status. Response: `ID` , `ExternalOrderNumber` , and `OrderStatus` with numeric `ID` and `Name` .

ID	STATUS
1	NewOrder
2	Cancelled
3	Delivered
4	PackingSlipPrinted
5	ChangeOrder
6	PartiallyDelivered
-1	Custom (shop-defined status)

POST /orders/{marketplace_instance_guid}

Creates a marketplace order in E37 from a `MarketplaceOrder` body. Returns 201 with `E37OrderNumber`, 400 on validation error, 500 on general error. Not needed for the read-only order database but relevant if marketplace integrations are ever consolidated.

POST /giftcard

Creates a gift card from a `GiftCardRequest`, returns a `GiftCardResponse`. Out of scope here.

Order status webhook

E37 can POST a JSON payload to a URL registered in E37 Admin whenever an order's status changes. Schema name in the spec: `Webhook-OrderStatusChanged`.

```
{
  "OrderID": 1189437,
  "ExternalOrderNumber": "",
  "ExternalOrderNumberExtra": "",
  "ErpOrderNumber": "",
  "OrderStatus": { "ID": 3, "Type": "Delivered", "Description": "Delivered" },
  "TrackingNumber": "",
  "TrackingURL": ""
}
```

FIELD	DESCRIPTION
<code>OrderID</code>	E37 order number
<code>ExternalOrderNumber</code>	External order number for marketplace orders
<code>ExternalOrderNumberExtra</code>	Order number at a marketplace hub such as Sello
<code>ErpOrderNumber</code>	Order number in the ERP
<code>OrderStatus.ID</code>	Same numeric codes as the status endpoint
<code>OrderStatus.Type</code> , <code>OrderStatus.Description</code>	Text representations
<code>TrackingNumber</code> , <code>TrackingURL</code>	From the TMS once shipped

The spec does not describe webhook authentication, signing or retry behaviour. Where in E37 Admin the URL is registered is also undocumented publicly.

Suggested integration

1. **Poll the order flow report** every 30 minutes per shop account, previous half-hour window, `orderTimestampMode=completed` . Store the rows as the order header table.
2. **Enrich per order** with `GET /orders/{id}` for customer, addresses, payment, delivery and rows. Store rows in an order-lines table keyed on order ID and row ID.
3. **Subscribe to the status webhook** to build status history and capture tracking numbers, instead of polling `/orders/{id}/status` .
4. **Backfill history** through E37's offered one-off export. The report endpoint may also accept long date intervals, which would allow a slow self-serve backfill; volume limits are **UNVERIFIED** .
5. **Reconcile** daily by re-fetching the previous day's windows and comparing counts, to catch any window lost to an outage.

Known shops and sites

From E37's report screenshot and recent correspondence. The `account` slug for each is only confirmed for Cykloteket.

SITE_ID	SITE_NAME	CURRENCY SEEN	NOTES
1	Cykloteket	SEK	<code>account=cykloteket</code> in E37's example
4	Bikester SE	SEK	
5	Bikester DK	DKK	
7	Bikester NO	NOK	
9	Birk Sport	NOK	

Separately, E37 confirmed on 2 September 2026 that `vartex.se` and `masterfitness.se` share one order number series in E37, and that a Svea Checkout session reserves an order number when a customer reaches checkout. Reserved numbers that never complete explain gaps in the sequence. Other WeSports shops on E37 (Vartex Outdoor, Outdoorexperter, Addnature, Larunpyörä) will each need their own account slug and possibly their own key.

Open questions for E37

1. API key creation in E37 Admin fails with an error. What is the correct path, and is a permission missing on our user?
2. Is one key valid across all WeSports accounts and sites, or is it per webshop ID?
3. Can the report endpoint accept the key via Basic auth header, like the REST API?
4. What timezone are `order_timestamp` and the `dateInterval` filter in? Are interval boundaries inclusive at both ends?
5. Are there rate limits, a maximum interval length or a maximum row count per call?
6. What are the JSON field names for the optional extra columns, and do site and status filters appear as query parameters?
7. What other values does `orderTimestampMode` accept?
8. Where is the status webhook URL registered, and is the payload signed?
9. Scope and timing of the one-off historical export: full tables, or the report endpoint over long intervals?
10. Which server, admin2 or admin3, hosts each WeSports shop?

Sources

- Email from Fredrik Karlsson, CEO and database architect at E37 System AB, 27 August 2026, "Sv: Engångsexport av orderhistorik – WeSports butiker", with six screenshots of the report UI, table output, JSON output and generated URL.
- Email from Niklas Hammar, WeSports, 2 August 2026, original request for a full order history export.
- Email from Fredrik Karlsson, 2 September 2026, "Sv: Hopp i orderserien", on shared order number series and Svea reservations.
- OpenAPI 3 specification "Triton Admin 1.0", admin3.e37.se/docs/Triton-Admin-1.0.json, rendered at admin3.e37.se/docs.
- E37 help centre, support.e37.se, searched for API key documentation on 3 September 2026. No article covers creating E37's own API key.

Compiled for the WeSports central order database project by Tommy Ivarsson, Vortex. Vendor: E37 System AB, support@e37.se. Update this page when E37 answers the open questions or when the first live call succeeds.